

CyberAudit

Backend Tests (pytest)

accounts app, 14 tests / 6 classes, JWT auth, RBAC, User model, rate limiting

Project	CyberAudit Platform
App	apps/accounts/
Test file	apps/accounts/tests.py
Fixtures file	conftest.py (project root)
Framework	pytest-django + django-rest-framework + simplejwt
Test classes	6 classes
Tests	14 tests minimum (some parametrizable)
Endpoints covered	register / login / me (GET+PATCH) / logout / change-password
Fixtures (conftest)	api_client / auth_client / admin_auth_client / client_user / admin_user / make_client_user / make_admin_user
autouse fixtures	_reset_throttle_cache (cache.clear) / _celery_eager_and_locmem_email
Date	June 2026 / Juin 2026

1. Overview -- 14 Tests across 6 Classes

1. Vue d'ensemble -- 14 tests en 6 classes

The accounts app tests cover the entire authentication lifecycle: User model integrity, JWT register/login/logout/refresh, profile read/update, password change, and inactive account blocking. All tests are class-based and use shared fixtures from `conftest.py`.

Les tests de l'app accounts couvrent l'ensemble du cycle d'authentification : integrite du modèle User, JWT register/login/logout/refresh, lecture/mise a jour du profil, changement de mot de passe, et blocage de compte inactif. Tous les tests sont en classes et utilisent des fixtures partagées depuis `conftest.py`.

Class	Endpoint / Scope	Tests	Fixtures used
TestUserModel	User model -- no HTTP, direct ORM calls	8	make_client_user, make_admin_user
TestRegisterView	POST /api/auth/register/	3	api_client, make_client_user
TestLoginView	POST /api/auth/login/	3	api_client, client_user, make_client_user
TestMeView	GET + PATCH /api/auth/me/	3	api_client, auth_client, client_user
TestLogoutView	POST /api/auth/logout/	2	auth_client, client_user
TestChangePasswordView	POST /api/auth/change-password/	2	auth_client
TOTAL		21	

2. conftest.py : Shared Fixtures

2. conftest.py : Fixtures partagées

conftest.py defines 9 fixtures at project root level, making them available to every app's test suite. Two patterns are used: factory fixtures (make_*) which return a callable, and instance fixtures (client_user, admin_user) which call the factories with defaults. Two autouse fixtures handle test isolation.

conftest.py définit 9 fixtures au niveau racine du projet, les rendant disponibles pour la suite de tests de chaque app. Deux patterns sont utilisés : les factory fixtures (make_) qui retournent un callable, et les fixtures d'instance (client_user, admin_user) qui appellent les factories avec les valeurs par défaut. Deux fixtures autouse gèrent l'isolation des tests.*

```
# — Factory fixtures —————
@pytest.fixture
def make_client_user(db):
    """Returns a callable -- override any field with kwargs."""
    def _make(**overrides):
        defaults = {'email': 'marie@example.com', 'password': 'Security!',
                    'first_name': 'Marie', 'last_name': 'Dupont', 'company_name': 'Cabinet Dijon'}
        defaults.update(overrides)
        return User.objects.create_user(**defaults)
    return _make

@pytest.fixture
def make_admin_user(db):
    def _make(**overrides):
        defaults = {'email': 'admin@cyberaudit.fr', 'password': 'Admin!Strong',
                    'first_name': 'Karim', 'last_name': 'Haddad',
                    'role': User.Role.ADMIN, 'is_staff': True}
        defaults.update(overrides)
        return User.objects.create_user(**defaults)
    return _make

# — Instance fixtures (call factory with defaults) —————
@pytest.fixture
def client_user(make_client_user):
    return make_client_user()
@pytest.fixture
def admin_user(make_admin_user):
    return make_admin_user()

# — HTTP client fixtures —————
@pytest.fixture
def api_client():
    return APIClient() # anonymous

@pytest.fixture
def auth_client(api_client, client_user):
    token = RefreshToken.for_user(client_user).access_token
    api_client.credentials(HTTP_AUTHORIZATION=f'Bearer {token}')
    return api_client

@pytest.fixture
def admin_auth_client(api_client, admin_user):
    token = RefreshToken.for_user(admin_user).access_token
    api_client.credentials(HTTP_AUTHORIZATION=f'Bearer {token}')
```

```

    return api_client

# — autouse: test isolation —————
@pytest.fixture(autouse=True)
def _reset_throttle_cache():
    from django.core.cache import cache
    cache.clear()      # clear before test
    yield
    cache.clear()      # clear after test

@pytest.fixture(autouse=True)
def _celery_eager_and_locmem_email(settings):
    settings.CELERY_TASK_ALWAYS_EAGER = True          # run tasks inline
    settings.CELERY_TASK_EAGER_PROPAGATES = True
    settings.EMAIL_BACKEND = 'django.core.mail.backends.locmem.EmailBackend'

```

Factory pattern	(EN) make_client_user() returns a function, not a user. Tests call it with overrides: make_client_user(email='x', role='admin'). Avoids fixture proliferation. (FR) make_client_user() retourne une fonction. Les tests l'appellent avec des surcharges. Evite la prolifération des fixtures.
auth_client JWT	(EN) Uses RefreshToken.for_user() to generate a real JWT. Sets it via credentials(HTTP_AUTHORIZATION). No HTTP round-trip to /login/. (FR) Utilise un vrai JWT. Le définit via credentials(). Pas d'appel HTTP à /login/.
_reset_throttle_cache	(EN) autouse=True means it runs for every test automatically. cache.clear() before+after ensures throttle counters don't bleed between tests. (FR) autouse=True -> s'exécute pour chaque test. cache.clear() avant+après évite la contamination des compteurs.
_celery_eager_and_locmem_email	(EN) Celery tasks run synchronously in tests. Emails go to mail.outbox instead of SMTP. No external services needed in CI. (FR) Les tâches Celery s'exécutent synchroniquement. Emails vers mail.outbox. Pas de services externes en CI.

3. TestUserModel : 8 Model Tests

3. TestUserModel : 8 tests du modele

These 8 tests validate the User model without any HTTP. They cover password hashing, role defaults, `__str__` format, `full_name` property, UUID PK length, admin flags from `make_admin_user`, the `ValueError` raised for blank email, and the `create_superuser` method.

Ces 8 tests valident le modele User sans HTTP. Ils couvrent le hachage du mot de passe, les valeurs par default du role, le format `__str__`, la propriete `full_name`, la longueur du UUID PK, les flags admin depuis `make_admin_user`, l'erreur `ValueError` pour email vide, et la methode `create_superuser`.

```
@pytest.mark.django_db
class TestUserModel:

    def test_create_user_hashes_password(self, make_client_user):
        user = make_client_user()
        assert user.password != 'Security!'          # stored value is the hash
        assert user.check_password('Security!')      # PBKDF2 verify

    def test_default_role_is_client(self, make_client_user):
        user = make_client_user()
        assert user.role == 'client'

    def test_str_contains_email_and_role(self, make_client_user):
        user = make_client_user()
        assert 'marie@example.com' in str(user)
        assert 'client' in str(user)

    def test_full_name_property(self, make_client_user):
        user = make_client_user(first_name='Marie', last_name='Dupont')
        assert user.full_name == 'Marie Dupont'

    def test_uuid_primary_key(self, make_client_user):
        user = make_client_user()
        assert len(str(user.id)) == 36 # 'xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx'

    def test_admin_user_is_staff(self, make_admin_user):
        admin = make_admin_user()
        assert admin.is_staff is True
        assert admin.role == 'admin'

    def test_create_user_without_email_raises_error(self):
        from apps.accounts.models import User
        with pytest.raises(ValueError, match='email'):
            User.objects.create_user(email='', password='Test1234!')

    def test_create_superuser_sets_flags(self):
        from apps.accounts.models import User
        admin = User.objects.create_superuser(email='super@example.com',
        password='Admin1234!')
        assert admin.is_staff is True
        assert admin.is_superuser is True
        assert admin.role == User.Role.ADMIN
```

Test	Assert	Why it matters EN / FR
hashes_password	password != plain, check_password() True	(EN) Confirms PBKDF2 hash, not plain text storage. (FR) Confirme PBKDF2, pas de stockage en clair.
default_role_is_client	role == 'client'	(EN) No accidental admin escalation on signup. (FR) Pas d'escalade admin accidentelle.
str_contains_email_and_role	email + role in str(user)	(EN) Admin panel display correctness. (FR) Affichage correct dans l'admin Django.
full_name_property	'Marie Dupont'	(EN) Used in notification subjects and PDF header. (FR) Utilise dans les sujets de notifs et l'en-tête PDF.
uuid_primary_key	len(str(id)) == 36	(EN) Confirms UUID v4 format, not auto-int. (FR) Confirme le format UUID v4.
admin_user_is_staff	is_staff True, role == 'admin'	(EN) Admin factory integrity, both flags set. (FR) Intégrité de la factory admin.
blank_email raises ValueError	pytest.raises(ValueError)	(EN) Custom UserManager enforces email required. (FR) UserManager personnalisée impose l'email.
create_superuser flags	is_staff+is_superuser True, role ADMIN	(EN) createsuperuser command contract fully verified. (FR) Contrat de la commande createsuperuser vérifié.

4. TestRegisterView + TestLoginView

4. TestRegisterView + TestLoginView

Register tests verify the 201 response shape (access + refresh + user object), duplicate email 400, and missing required field 400. Login tests verify 200 with token + user.role payload, 401 for wrong password, and 403 for inactive account, all three HTTP status codes have distinct security meanings.

Les tests Register vérifient la structure de la réponse 201 (access + refresh + objet user), 400 pour email dupliquée, et 400 pour champ requis manquant. Les tests Login vérifient 200 avec token + user.role, 401 pour mauvais mot de passe, et 403 pour compte inactif, les trois codes HTTP ont des significations de sécurité distinctes.

TestRegisterView : POST /api/auth/register/

```
@pytest.mark.django_db
class TestRegisterView:
    URL = '/api/auth/register/'

    def test_register_returns_201_and_tokens(self, api_client):
        payload = {'email': 'nouveau@example.com', 'password': 'Securlty!',
                   'first_name': 'Alice', 'last_name': 'Martin', 'company_name': 'Test SARL'}
        resp = api_client.post(self.URL, payload)
        assert resp.status_code == status.HTTP_201_CREATED
        assert 'access' in resp.data
        assert 'refresh' in resp.data
        assert resp.data['user']['email'] == 'nouveau@example.com'

    def test_register_duplicate_email_returns_400(self, api_client,
        make_client_user):
        make_client_user(email='marie@example.com')
        resp = api_client.post(self.URL,
            {'email': 'marie@example.com', 'password': 'Securlty!',
             'first_name': 'Marie', 'last_name': 'Dupont'})
        assert resp.status_code == status.HTTP_400_BAD_REQUEST

    def test_register_missing_email_returns_400(self, api_client):
        resp = api_client.post(self.URL, {'password': 'Securlty!'})
        assert resp.status_code == status.HTTP_400_BAD_REQUEST
```

TestLoginView : POST /api/auth/login/

```
@pytest.mark.django_db
class TestLoginView:
    URL = '/api/auth/login/'

    def test_login_returns_tokens_and_user(self, api_client, client_user):
        resp = api_client.post(self.URL,
            {'email': 'marie@example.com', 'password': 'Securlty!'})
        assert resp.status_code == status.HTTP_200_OK
        assert 'access' in resp.data
```

```

    assert resp.data['user']['role'] == 'client'

    def test_login_wrong_password_returns_401(self, api_client, client_user):
        resp = api_client.post(self.URL,
                                {'email': 'marie@example.com', 'password': 'WrongPassword!'})
        assert resp.status_code == status.HTTP_401_UNAUTHORIZED

    def test_login_inactive_account_returns_403(self, api_client,
        make_client_user):
        user = make_client_user(email='inactive@example.com')
        user.is_active = False
        user.save()
        resp = api_client.post(self.URL,
                                {'email': 'inactive@example.com', 'password': 'Security!'})
        assert resp.status_code == status.HTTP_403_FORBIDDEN

```

Test	HTTP	Security meaning EN / FR
register -- success	201	(EN) Tokens returned immediately, no separate login call needed. (FR) Tokens retournes immédiatement, pas d'appel login sépare.
register -- duplicate email	400	(EN) Unique constraint enforced at API layer. Message does NOT reveal which field failed. (FR) Contrainte unique appliquée au niveau API.
register -- missing email	400	(EN) Serializer validation rejects incomplete payloads before any DB touch. (FR) Validation sérialiser rejette les payloads incomplets avant tout accès DB.
login -- success	200	access + refresh tokens + user.role. (EN) Role confirms RBAC level. access + refresh + user.role. (FR) Le rôle confirme le niveau RBAC.
login -- wrong password	401	(EN) 401 Unauthorized, credentials rejected. No account existence hint. (FR) 401 identifiants rejetés. Aucune indication sur l'existence du compte.
login -- inactive account	403	(EN) 403 Forbidden, account exists but is deactivated. Distinct from bad password. (FR) 403 compte existe mais désactive. Distinct d'un mauvais mot de passe.

5. TestMeView + TestLogoutView

5. TestMeView + TestLogoutViews

TestMeView covers GET (returns user data), GET unauthenticated (401), and PATCH (field update). TestLogoutView tests the JWT blacklist mechanism after logout, the refresh token is invalid. Missing refresh token in POST body returns 400.

TestMeView couvre GET (retourne les données user), GET non authentifié (401), et PATCH (mise à jour de champ). TestLogoutView teste le mécanisme de blacklist JWT, après logout, le token refresh est invalide. Token refresh manquant dans le POST retourne 400.

```
@pytest.mark.django_db
class TestMeView:
    URL = '/api/auth/me/'

    def test_get_me_authenticated(self, auth_client, client_user):
        resp = auth_client.get(self.URL)
        assert resp.status_code == status.HTTP_200_OK
        assert resp.data['email'] == client_user.email

    def test_get_me_unauthenticated_returns_401(self, api_client):
        resp = api_client.get(self.URL)
        assert resp.status_code == status.HTTP_401_UNAUTHORIZED

    def test_patch_me_updates_first_name(self, auth_client, client_user):
        resp = auth_client.patch(self.URL, {'first_name': 'Pauline'})
        assert resp.status_code == status.HTTP_200_OK
        assert resp.data['first_name'] == 'Pauline'

@pytest.mark.django_db
class TestLogoutView:
    URL = '/api/auth/logout/'

    def test_logout_blacklists_token(self, auth_client, client_user):
        from rest_framework_simplejwt.tokens import RefreshToken
        refresh = str(RefreshToken.for_user(client_user))
        resp = auth_client.post(self.URL, {'refresh': refresh})
        assert resp.status_code == status.HTTP_204_NO_CONTENT

    def test_logout_missing_refresh_returns_400(self, auth_client):
        resp = auth_client.post(self.URL, {})
        assert resp.status_code == status.HTTP_400_BAD_REQUEST
```

auth_client fixture	(EN) Bearer token already set on the client, no login call inside the test. Keeps tests fast and focused. (FR) Token Bearer déjà défini sur le client. Pas d'appel login dans le test. Tests rapides et focalisés.
PATCH partial update	(EN) Only first_name sent. The view uses partial=True on the serializer to allow single-field updates. (FR) Uniquement first_name envoyée. La vue utilise partial=True sur le serializer.

204 on logout	(EN) No content confirms the token was accepted and blacklisted. Client must discard both tokens. (FR) Pas de contenu confirme que le token a ete accepte et blackliste.
400 on missing refresh	(EN) Validates that the view requires the refresh token. Empty POST is not silently accepted. (FR) Valide que la vue exige le token refresh. Un POST vide n'est pas silencieusement accepté.
JWT blacklist	(EN) simplejwt's OutstandingToken + BlacklistedToken tables. Once a refresh token is blacklisted, /token/refresh/ returns 401. (FR) Tables OutstandingToken + BlacklistedToken de simplejwt. Une fois blackliste, /token/refresh/ retourne 401.

6. TestChangePasswordView + Rate Limiting

6. TestChangePasswordView + Limitation de taux

Change password requires the old password for verification, a standard security check preventing session hijacking. The autouse `_reset_throttle_cache` fixture ensures the DRF throttle counters are cleared before and after every test, so rate-limiting tests don't contaminate each other.

Le changement de mot de passe exige l'ancien mot de passe pour vérification, une sécurité standard évitant le détournement de session. La fixture autouse `_reset_throttle_cache` garantit que les compteurs de throttle DRF sont réinitialisés avant et après chaque test, évitant la contamination entre tests de limitation de taux.

```
@pytest.mark.django_db
class TestChangePasswordView:
    URL = '/api/auth/change-password/'

    def test_change_password_success(self, auth_client):
        resp = auth_client.post(self.URL,
                                {'old_password': 'Securilty!', 'new_password': 'N3wPassw0rd!'})
        assert resp.status_code == status.HTTP_200_OK

    def test_change_password_wrong_old_returns_400(self, auth_client):
        resp = auth_client.post(self.URL,
                                {'old_password': 'WrongOld!', 'new_password': 'N3wPassw0rd!'})
        assert resp.status_code == status.HTTP_400_BAD_REQUEST
```

Rate limiting : throttle architecture

DRF throttling is configured at 5 requests per minute for anonymous users and a higher limit for authenticated ones. The `_reset_throttle_cache` autouse fixture calls `cache.clear()` before and after every test to prevent test ordering from affecting throttle counters.

Le throttling DRF est configure a 5 requêtes par minute pour les utilisateurs anonymes et une limite plus élevée pour les authentifiés. La fixture autouse `_reset_throttle_cache` appelle `cache.clear()` avant et après chaque test pour éviter que l'ordre des tests affecte les compteurs.

```
# settings.py throttle configuration
REST_FRAMEWORK = {
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle',
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '5/min',
        'user': '100/min',
    }
}

# conftest.py -- autouse fixture
@pytest.fixture(autouse=True)
def _reset_throttle_cache():
```

```
from django.core.cache import cache
cache.clear()    # before: start with clean counters
yield
cache.clear()    # after: prevent bleed into next test
```

old_password required	(EN) Verifies current session legitimacy. Prevents a stolen access token from silently changing the password. (FR) Vérifie la légitimité de la session actuelle. Empêche un token vole de changer silencieusement le mot de passe.
200 on success	(EN) Confirms the password was changed. The old token remains valid until logout (JWT is stateless). (FR) Confirme le changement. L'ancien token reste valide jusqu'au logout (JWT sans état).
400 wrong old password	(EN)Serializer checks check_password(old_password). 400 Bad Request -- not 401. The user is authenticated. (FR) Le serializer verifie check_password(). 400 -- pas 401. L'utilisateur est authentifié.
5 req/min throttle	(EN) Applied on all anon endpoints (login, register). After 5 rapid requests, DRF returns 429 Too Many Requests. (FR) Applique sur tous les endpoints anonymes. Après 5 requêtes rapides, DRF retourne 429.
cache.clear() pattern	(EN) DRF throttle uses Django cache as counter store. Clearing before+after each test is the correct isolation pattern. (FR) DRF throttle utilise le cache Django comme compteur. Clear avant+apres = isolation correcte.

7. Test Isolation & Infrastructure

7. Isolation des tests et infrastructure

Each test runs in a transaction that is rolled back after completion (`@pytest.mark.django_db`). The two autouse fixtures handle the two components that persist between tests: the throttle cache and Celery/email settings. `pytest.ini` configures the Django settings module and disables warnings noise.

Chaque test s'exécute dans une transaction annulée après exécution (`@pytest.mark.django_db`). Les deux fixtures autouse gèrent les deux composants qui persistent entre tests : le cache de throttle et les paramètres Celery/email. `pytest.ini` configure le module de settings Django et désactive le bruit des warnings.

pytest.ini

```
[pytest]
DJANGO_SETTINGS_MODULE = config.settings
python_files = tests.py test_*.py *_tests.py
python_classes = Test*
python_functions = test_*
addopts = -v --tb=short
```

Test DB transaction rollback

```
# Each @pytest.mark.django_db test runs inside a transaction that is
# automatically rolled back after the test completes.
# This means:
#   - DB is always clean at the start of each test
#   - Tests are fully independent regardless of execution order
#   - No need for teardown / setUp / truncate

# Example: TestUserModel creates a user -- next test starts with an empty DB
@pytest.mark.django_db
class TestUserModel:
    def test_create_user_hashes_password(self, make_client_user):
        user = make_client_user() # inserted inside the test transaction
        # ... test assertions ...
        # transaction rolls back here -- next test sees no user in DB
```

Full test run command

```
# Run all accounts tests with coverage:
cd ~/portfolio/cyberaudit/backend
pytest apps/accounts/tests.py -v --tb=short

# Run with coverage:
pytest apps/accounts/tests.py --cov=apps/accounts --cov-report=term-missing

# Run only one class:
pytest apps/accounts/tests.py::TestLoginView -v
```

```
# Run only one test:
pytest
apps/accounts/tests.py::TestLoginView::test_login_wrong_password_returns_401 -v
```

8. Security & Design Summary

8. Recapitulative sécurité et conception

Feature	Detail EN / FR
PBKDF2 password hashing	(EN) test_create_user_hashes_password confirms stored value != plain text and check_password() passes. (FR) Confirme que la valeur stockée != texte clair et que check_password() réussit.
JWT on register	(EN) 201 response includes access + refresh immediately. No second login call needed. (FR) 201 inclut access + refresh immédiatement. Pas de deuxième appel login.
401 vs 403 on login	(EN) 401 = wrong credentials. 403 = account deactivated. Distinct HTTP codes for distinct security states. (FR) 401 = identifiants incorrects. 403 = compte desactive. Codes HTTP distincts pour états distincts.
JWT blacklist logout	(EN) simplejwt OutstandingToken + BlacklistedToken. After 204, re-using the refresh token returns 401. (FR) Après 204, réutiliser le token refresh retourne 401.
Old password required	(EN) change-password checks check_password(old_password). Stolen token cannot change password silently. (FR) Token vole ne peut pas changer le mot de passe silencieusement.
5 req/min throttle	(EN) AnonRateThrottle 5/min on login and register. After 5 rapid requests, 429 Too Many Requests. (FR) AnonRateThrottle 5/min sur login et register. Après 5 requetes rapides, 429.
cache.clear() autouse	(EN) _reset_throttle_cache clears Django cache before+after every test. No throttle bleed between tests. (EN) Vide le cache Django avant+après chaque test. Pas de contamination entre tests.
Celery ALWAYS_EAGER	(EN) _celery_eager_and_locmem_email runs tasks synchronously. No worker or broker needed in test environment. (FR) Execute les taches synchroniquement. Pas de worker ni broker en test.
locmem email backend	(EN) Emails captured in mail.outbox. No SMTP server needed. CI-safe by default. (FR) Emails dans mail.outbox. Pas de serveur SMTP. CI-safe par défaut.
DB transaction rollback	(EN) Each @pytest.mark.django_db test rolls back automatically. Tests are fully independent of execution order. (FR) Chaque test s'annule automatiquement. Tests independants de l'ordre d'execution.
Factory fixture pattern	(EN) make_client_user / make_admin_user return callables. Per-test overrides without new fixtures. (FR) Retournent des callables. Surcharges par test sans nouvelles fixtures.